Football Statistics API Documentation (v1.3)

Base Information

- Base URL: `http://localhost:3000`
- Swagger UI: `http://localhost:3000/api-docs`
- API Version: `1.3.0`

Authentication and Authorization

The API uses JWT bearer tokens.

- Public: all `GET` endpoints
- Auth required: all `POST` and `PUT` endpoints
- Admin required: all `DELETE` endpoints on Teams, Players, and Matches

Header format:

```
Authorization: Bearer <jwt_token>
```

Global Middleware and API Rules

Security

- `helmet` is enabled for HTTP security headers.
- `express-rate-limit` is enabled globally.
  - Window: 15 minutes
  - Max requests per IP: 200
  - Exceeded response:

```
{ "error": "Too many requests. Please try again later." }
```

Pagination

List endpoints share one pagination middleware.

- Default page: `1`
- Default limit: `20`
- Maximum limit: `100`

If a larger `limit` is sent, it is automatically capped to `100`.

Error Format

```
{
  "error": "Human-readable message",
```

```
  "details": ["optional field-level validation details"]
}
```

Common status codes:

- `200` success
- `201` created
- `400` validation or bad request
- `401` unauthenticated
- `403` unauthorized (role)
- `404` not found
- `409` conflict
- `429` rate-limited
- `500` internal error

Endpoint Summary

Authentication

Method	Endpoint	Auth
POST	`/api/auth/register`	No
POST	`/api/auth/login`	No
GET	`/api/auth/profile`	Yes

Teams

Method	Endpoint	Auth
GET	`/api/teams`	No
GET	`/api/teams/:id`	No
POST	`/api/teams`	Yes
PUT	`/api/teams/:id`	Yes
DELETE	`/api/teams/:id`	Admin

Players

Method	Endpoint	Auth
GET	`/api/players`	No
GET	`/api/players/:id`	No
POST	`/api/players`	Yes

PUT	`/api/players/:id`	Yes
DELETE	`/api/players/:id`	Admin

Matches

Method	Endpoint	Auth
GET	`/api/matches`	No
GET	`/api/matches/:id`	No
POST	`/api/matches`	Yes
PUT	`/api/matches/:id`	Yes
DELETE	`/api/matches/:id`	Admin

Analytics

Method	Endpoint	Description
GET	`/api/analytics/leaderboard`	League standings by season + competition
GET	`/api/analytics/top-scorers`	Top players by goals and assists
GET	`/api/analytics/teams/:id/performance`	Team performance summary
GET	`/api/analytics/head-to-head`	Team-vs-team comparison
GET	`/api/analytics/seasons/:season`	Season totals and trends
GET	`/api/analytics/teams/:id/form-trend`	Rolling form and cumulative points
GET	`/api/analytics/teams/:id/home-away`	Home vs away split and advantage

Key Request/Response Notes

POST `/api/auth/register`

Request:

```
{
  "username": "john_doe",
  "email": "john@example.com",
  "password": "securePass123"
}
```

Notes:

- Role is always assigned as `user`.
- Returns user object and JWT token.

GET `/api/teams`

Optional query params: `page`, `limit`, `country`, `league`, `search`

Response includes:

- `teams`: array
- `pagination`: `{ total, page, limit, totalPages }`

GET `/api/players`

Optional query params: `page`, `limit`, `position`, `nationality`, `teamId`, `search`

GET `/api/matches`

Optional query params: `page`, `limit`, `season`, `competition`, `status`, `teamId`

GET `/api/analytics/leaderboard`

Required query params:

- `season`
- `competition`

Returns computed standings with points, goal difference, and ranking.

GET `/api/analytics/top-scorers`

Optional query params:

- `limit` (default 10)
- `position`
- `teamId`

GET `/api/analytics/teams/:id/performance`

Optional query params:

- `season`
- `competition`

Returns `stats` and `recentForm`.

GET `/api/analytics/head-to-head`

Required query params:

- `team1Id`
- `team2Id`

Returns wins/goals summary plus recent matches.

GET `/api/analytics/seasons/:season`

Returns:

- total matches/goals
- home/away/draw breakdown
- highest scoring match
- monthly goals trend

GET `/api/analytics/teams/:id/form-trend` **(new in v1.2)**

Optional query params:

- `season`
- `competition`
- `last` (default 20, capped to 50)

Returns:

- summary (`pointsPerGame` , `avgScored` , `avgConceded` , `formString`)
- `trend` array with cumulative points after each match

GET `/api/analytics/teams/:id/home-away` **(new in v1.2)**

Optional query params:

- `season`
- `competition`

Returns:

- `home` and `away` split objects
- comparison metrics including `homeAdvantage`

Data Import and Local Setup

Install and run:

```
npm install
npm run import   # import Kaggle CSV from data/results.csv
npm start
```

Alternative seed:

```
npm run seed
```

Testing

```
npm test
npm run test:coverage
```

Current v1.3 test profile:

- 56 integration tests
- 82.68% statement coverage

PDF export command:

```
npm run docs:pdf
```